Pioneer Delivery

PART II / Task D: Validation and algorithm analysis

A. Closest delivery locations

The algorithm sorts the points once by x and once by y, then divides the x-list into balanced halves. It partitions the y-list using point IDs, so points with the same x-coordinate still go into the correct half. Each recursive call returns a closest pair and its squared distance. The base case checks every pair only when there are at most three points.

The smaller result from the two halves gives the current distance bound. A better cross-half pair must be in the strip around the dividing line. The strip is already in y-order, and each point checks at most the next seven points. The packing bound comes from the minimum separation within each half: two delta-wide half-strips of height delta can each be split into four small cells, with at most one point from that half per cell. This limits the candidates rather than comparing every pair. Every pair is either inside a half or crosses the split, so recursion plus the strip covers a closest pair.

Filtering the y-list and checking the strip take O(n) per level. Therefore T(n) = 2T(n/2) + O(n), giving O(n log n) including the initial sorts. The temporary lists and ID sets along a recursive path total O(n) auxiliary space; stack depth is O(log n).

B. Dispatch-order inversions

A lookup table maps IDs to their positions in the planned order. Actual IDs become an array of those ranks. Merge sort recursively counts inversions in each half. While merging, choosing a smaller right value adds len(left) - i, because that value is smaller than every remaining left value. This counts each cross-half inversion once; together with the recursive counts, every inverted pair is counted exactly once.

The rank mapping uses standard hash-table lookups. The merge recurrence is T(n) = 2T(n/2) + O(n), so the algorithm takes O(n log n) time and O(n) auxiliary space, including merge lists. Stack depth is O(log n). Python integers represent n(n - 1)/2 without a 32-bit overflow.

C. Delivery-time selection

Groups of at most five values are sorted to get their medians. The algorithm recursively selects the median of those medians as the pivot. It partitions into lower, equal, and higher values. For 1-based rank k, recurse into lower with the same k, return the pivot if k falls in equal, or recurse into higher with k - len(lower) - len(equal). Only the partition containing the answer is kept, so repeated values are handled as separate ranks.

The full groups provide at least 3n/10 - O(1) values on each non-strict side of the pivot. Thus a recursive selection partition has at most 7n/10 + O(1) values. T(n) <= T(ceil(n/5)) + T(7n/10 + O(1)) + O(n) is O(n) worst case. Sorting is restricted to groups and base cases of at most five. The list-based implementation uses O(n) auxiliary space and O(log n) stack depth.

Testing and performance

Required cases, extra checks, and measured execution times

Correctness checks

All 22 saved cases pass, including the 13 numbered required cases, all requested extensions, the 100,000-item inversion check, and extra cases for each task. The automated suite passes 34 tests. main.py --cases prints each expected and actual result and saves the output. tests/test_project.py also compares against independent references on 200 small inputs per task: every point pair for A, nested loops for B, and sorting for C. For C, every valid rank is checked on each small input.

Extra edge cases include all x-coordinates tied with the closest pair across the split, large integer coordinates, nonalphabetical planned IDs, and repeated zero delivery times. A tied closest-pair answer is accepted if the IDs produce the minimum squared distance. B returns 4,999,950,000 for 100,000 reversed IDs. Invalid inputs, such as fewer than two locations, duplicate coordinates or IDs, mismatched dispatch orders, negative times, and out-of-range k, raise clear errors. Tests also verify unchanged input lists, exactly two initial sorts for A, and no sort larger than five values for C.

Performance method

These are measured results on Darwin arm64 using cpython 3.9.6. For each input size, benchmark.py uses seed 601 + n to create distinct random points, shuffled dispatch ranks, and integer times with duplicates. Input generation is outside the timer. Each function gets one warm-up, then seven perf_counter measurements; the table shows the median in milliseconds. Public-function input validation is included. All individual samples are saved in results/timings.json.

Measured median times

Input size n	A: closest pair	B: inversions	C: selection
100	0.3825 ms	0.1388 ms	0.0173 ms
1,000	4.6102 ms	1.3072 ms	0.1598 ms
5,000	25.2703 ms	7.9541 ms	1.2615 ms

Growth and reproduction

From n = 1,000 to 5,000, A grows 5.48 times and B grows 6.08 times. The corresponding n log2(n) factor is 6.16, so these measurements are broadly consistent with their expected growth. C grows 7.89 times over the same fivefold input increase, rather than exactly five times. Specific partition balance, duplicates, allocation costs, and short measurement times affect the constants. The timing samples do not prove a worst-case bound; C's deterministic pivot recurrence supplies that guarantee.

Run from Part-II: python3 main.py --cases; python3 -m unittest discover -s tests -v; python3 benchmark.py --sizes 100 1000 5000 --repeats 7. Re-running the benchmark updates timings.json but does not change the numbers recorded in this report. Timings vary by machine and system load.